TestPulse Dual-Surface Architecture

IDE AI + MCP tools synchronized with a React Flow visualization UI

Design correction: React Flow is not between the IDE and MCP. It is a separate visualization client over the same TestPulse intelligence layer.

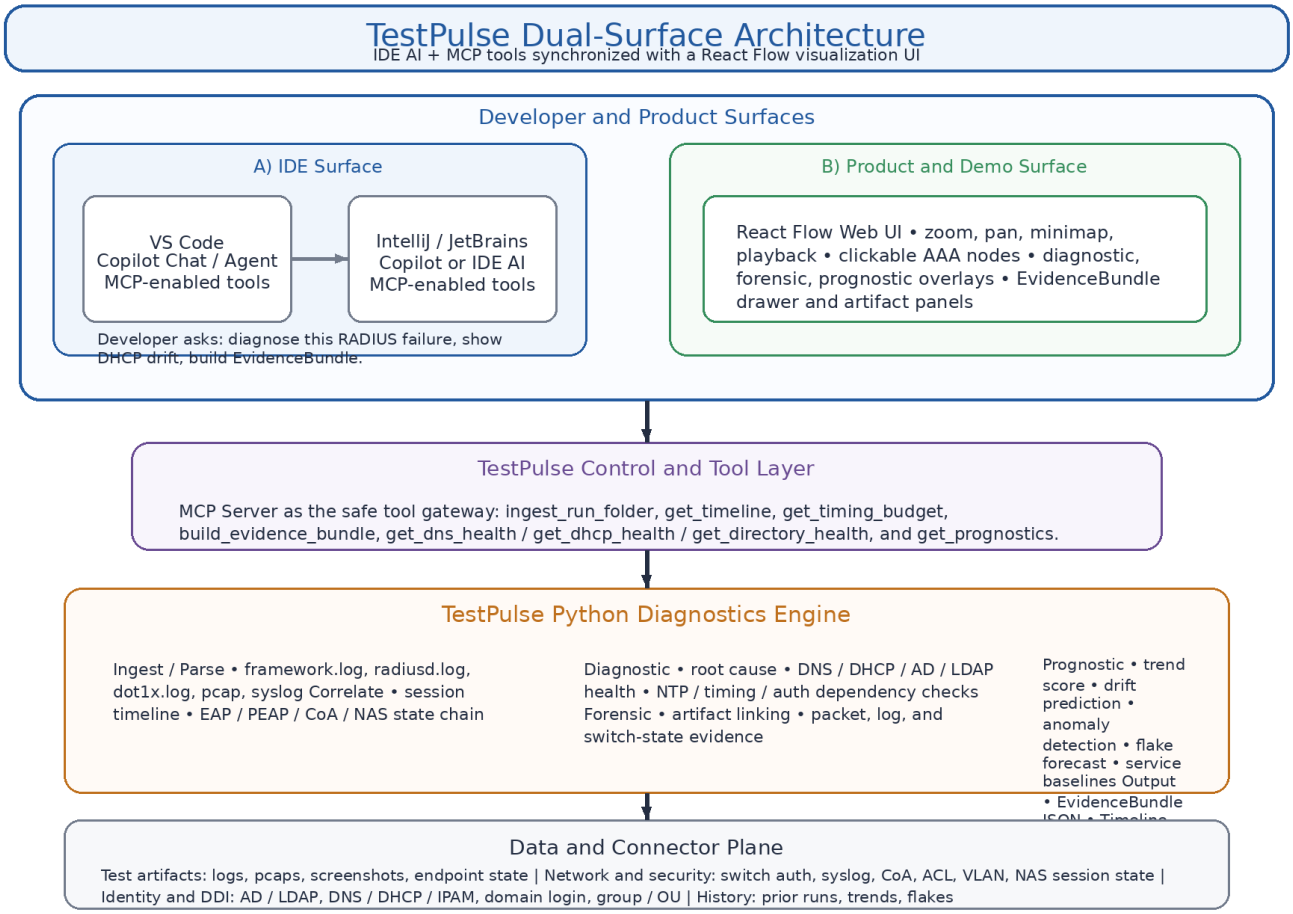

Figure 1. Dual-surface architecture for the IDE experience, product UI, MCP layer, diagnostics engine, and shared data plane.

Commercialization story

- Inside the IDE, developers use Copilot or IDE AI to ask questions and run diagnostics without raw log digging.
- Inside the product UI, QA, operations, support, and customers see a zoomable, clickable AAA map with evidence-backed diagnostics, forensics, and prognostics.
- Inside the platform core, the same backend produces timelines, health signals, and EvidenceBundles for every surface.

Synchronization view

Synchronization means the same request context, the same run ID and session ID, and the same EvidenceBundle truth, even though the results are consumed by different user surfaces.

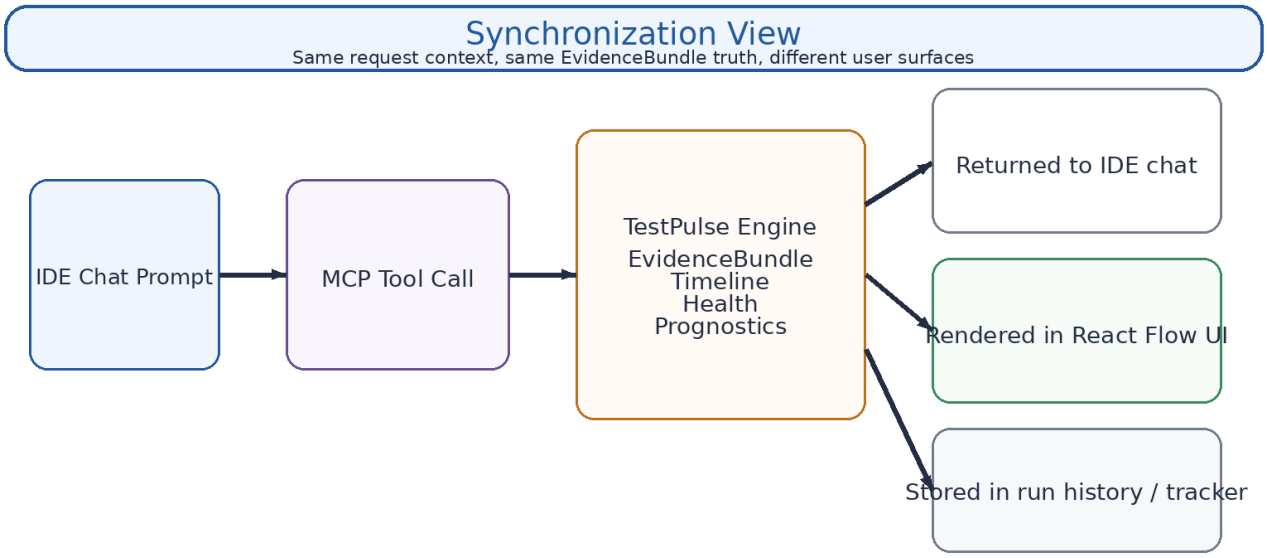

Figure 2. Copilot or IDE AI calls MCP tools, the TestPulse engine produces shared outputs, and those outputs are returned to chat, rendered in React Flow, and stored for history and tracker workflows.

Recommended component mapping

Layer	Primary choice	Why it matters
IDE layer	VS Code with GitHub Copilot Chat and MCP tools first; IntelliJ / JetBrains added next	Gives developers an in-context AI workflow tied to the same diagnostics engine.
Visualization layer	React Flow	Creates the zoomable, clickable, demo-friendly AAA system map that Mermaid cannot provide at runtime.
Tool layer	MCP	Keeps AI clients on a safe tool rail for timeline retrieval, health checks, EvidenceBundle generation, and prognostic scoring.
Engine layer	Python TestPulse backend	Centralizes ingest, correlation, diagnostics, forensics, prognostics, and bundle creation.

Phase roadmap

Phase 1

- VS Code + GitHub Copilot + MCP
- React Flow web UI
- Python TestPulse backend
- Single MCP server

Phase 2

- Add JetBrains / IntelliJ
- Optionally support JetBrains AI Assistant as a second chat front end
- Keep the same MCP-backed TestPulse services

Phase 3

- Expose a customer-facing dashboard
- Add reporting, export, and tracker sync
- Make prognostics a visible early-warning panel

One-line architecture summary

TestPulse is a dual-surface platform: developers interact through IDE AI chat and MCP tools, while operators and customers use a React Flow visualization UI. Both are synchronized through the same TestPulse diagnostics engine, which produces shared timelines, health signals, and EvidenceBundles.